

Instituto Tecnológico Superior de Rio verde SLP.

TEMA:

Semana 9

ALUMNO:

Brayan Ivanov Martínez Martínez

MATERIA:

programacion

DOCENTE:

Jesus collazo

GRUPO

Fecha
15/04/2026

¿Qué versiones del método mover existen?

`void mover(); void mover(char direccion); void mover(int pasos);`

¿Cuál se ejecuta en cada caso?

`p.mover(); p.mover('d'); p.mover(3);`

¿Cómo decide el programa qué función usar?

El compilador decide usando: el tipo y número de parámetros
Sin parámetros `mover()` Char `mover(char)` Int `mover(int)`

¿Qué hace el operador `+` en este programa?

Crea una copia del objeto (`temp`), Suma puntos a la copia,
Regresa el nuevo objeto

¿Qué hace el operador `+=`?

Modifica el objeto directamente y no crea copia

¿Qué cambia en el objeto después de usar estos operadores?

`+`: crea nuevo objeto con más puntos

`+=`: modifica el objeto actual

Introducción

En esta práctica se realizó un programa en C++ donde se simula un personaje tipo Pacman y se probaron sus diferentes acciones.

El tema que se trabaja es la sobrecarga, que permite usar un mismo método o operador de distintas formas dependiendo de lo que se le pase.

El objetivo es entender cómo funciona la sobrecarga y cómo cambia el comportamiento del objeto dentro del programa.

```
C Pacman.h
1  #ifndef PACMAN_H
2  #define PACMAN_H
3
4  #include <iostream>
5  using namespace std;
6
7  class Pacman {
8
9  private:
10     int x, y;
11     int puntos;
12
13 public:
14     Pacman(int x, int y);
15
16     void mostrar();
17
18     // Sobrecarga de métodos
19     void mover();
20     void mover(char direccion);
21     void mover(int pasos);
22
23     // Sobrecarga de operadores
24     Pacman operator+(int p);
25     void operator+=(int p);
26 };
27
28 #endif
```

En esta captura se muestra la definición de la clase Pacman aquí se declaran los atributos como la posición (x, y) y los puntos, así como los métodos que el objeto puede realizar, incluyendo las diferentes versiones del método mover y los operadores sobrecargados este archivo funciona como la estructura base del programa.

```

1  #include "Pacman.h"
2
3  // Constructor
4  Pacman::Pacman(int x, int y) {
5      this->x = x;
6      this->y = y;
7      puntos = 0;
8
9      cout << "Pacman creado en posicion (" << x << "," << y << ")\n";
10 }
11
12 // Mostrar estado
13 void Pacman::mostrar() {
14     cout << "Posicion: (" << x << "," << y << ") | Puntos: " << puntos << endl;
15 }
16
17 // Sobrecarga 1
18 void Pacman::mover() {
19     x++;
20     cout << "Movimiento por defecto\n";
21 }
22
23 // Sobrecarga 2
24 void Pacman::mover(char direccion) {
25     if (direccion == 'w') y--;
26     else if (direccion == 's') y++;
27     else if (direccion == 'a') x--;
28     else if (direccion == 'd') x++;
29
30     cout << "Movimiento con direccion\n";
31 }
32
33 // Sobrecarga 3
34 void Pacman::mover(int pasos) {
35     x += pasos;
36     cout << "Movimiento multiple\n";
37 }
38
39 // Operador +
40 Pacman Pacman::operator+(int p) {
41     Pacman temp = *this;
42     temp.puntos += p;
43     return temp;
44 }
45
46 // Operador +=
47 void Pacman::operator+=(int p) {
48     puntos += p;
49 }

```

En esta captura se observa la implementación de los métodos de la clase Pacman aquí se define cómo se mueve el personaje en sus diferentes versiones así como el funcionamiento de los operadores + y += este archivo contiene la lógica que permite modificar el estado del objeto.

```

G+ main.cpp
1  #include "Pacman.h"
2
3  int main() {
4
5      Pacman p(0,0);
6      p.mostrar();
7
8      p.mover();
9      p.mostrar();
10
11     p.mover('d');
12     p.mostrar();
13
14     p.mover(3);
15     p.mostrar();
16
17     p = p + 10;
18     p.mostrar();
19
20     p += 5;
21     p.mostrar();
22
23     return 0;
24 }
25

```

En esta captura se muestra el programa principal donde se crea el objeto Pacman y se utilizan sus métodos aquí se realizan diferentes movimientos

```

ivano@ivanov MINGW64 ~/OneDrive/Escritorio/p1/semana9
$ g++ main.cpp Pacman.cpp -o programa

ivano@ivanov MINGW64 ~/OneDrive/Escritorio/p1/semana9
$ ./programa

```

En esta captura se observa el proceso de compilación del programa utilizando el comando g++ convierte el código fuente en un archivo ejecutable y sin errores.

```
ivano@ivanov MINGW64 ~/OneDrive/Escritorio/p1/semana9
$ g++ main.cpp Pacman.cpp -o programa

ivano@ivanov MINGW64 ~/OneDrive/Escritorio/p1/semana9
$ ./programa
Pacman creado en posicion (0,0)
Posicion: (0,0) | Puntos: 0
Movimiento por defecto
Posicion: (1,0) | Puntos: 0
Movimiento con direccion
Posicion: (2,0) | Puntos: 0
Movimiento multiple
Posicion: (5,0) | Puntos: 0
Posicion: (5,0) | Puntos: 10
Posicion: (5,0) | Puntos: 15

ivano@ivanov MINGW64 ~/OneDrive/Escritorio/p1/semana9
$
```

En esta captura se muestra la ejecución del programa ya compilado aquí se pueden observar los resultados en pantalla como los movimientos del Pacman y el cambio en sus puntos lo que confirma que el programa funciona correctamente.

Análisis

La sobrecarga de métodos es cuando un mismo método tiene varias formas, como mover() que cambia según lo que le mandes.

La sobrecarga de operadores es cuando operadores como + y += se usan con objetos. El + crea uno nuevo con más puntos y el += le suma puntos al mismo objeto.

Conclusiones

En esta práctica aprendí cómo funciona la sobrecarga en C++ tanto en métodos como en operadores y cómo esto permite que un mismo nombre tenga diferentes comportamientos según lo que se le mande.

También entendí mejor cómo trabaja un objeto dentro de un programa, ya que sus valores como la posición y los puntos van cambiando dependiendo de las acciones que se realizan.

Al principio fue un poco difícil identificar qué método se estaba ejecutando pero al revisar se vuelve más fácil entenderlo.

Análisis

En el programa existen varios métodos sobrecargados del método mover está mover() que mueve al personaje una posición mover(char direccion) que lo mueve según la dirección indicada (arriba, abajo, izquierda o derecha), y mover(int pasos) que lo mueve varias posiciones cada uno hace algo parecido pero cambia según el dato que recibe.

También se redefinen los operadores + y += el operador + crea un nuevo objeto con más puntos sin modificar el original, mientras que el operador += sí modifica directamente el objeto sumándole puntos.

El archivo Pacman.h contiene la definición de la clase y los métodos, el archivo Pacman.cpp tiene la lógica donde se programan los movimientos y operadores, y el archivo main.cpp es donde se crea el objeto y se ejecutan las acciones.

Todos los archivos se relacionan porque el main usa la clase definida en el .h y ejecuta las funciones que están implementadas en el .cpp.

El comportamiento del objeto cambia dependiendo del método u operador que se use ya que puede moverse de diferentes formas y aumentar sus puntos, mostrando resultados distintos en cada caso.